

Bases de Données, Introduction à l'optimisation

Chapitre 01: Présentation Générale

Asma BERRIRI

asma.berriri@universite-paris-saclay.fr

Page web du cours :

<https://sites.google.com/view/asmaberriri/home/aspects-systeme-base-de-donnees>

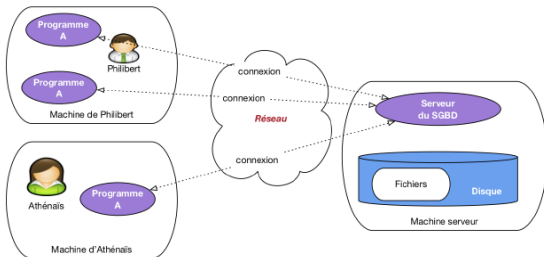
Table of Contents

1 Plan du Cours

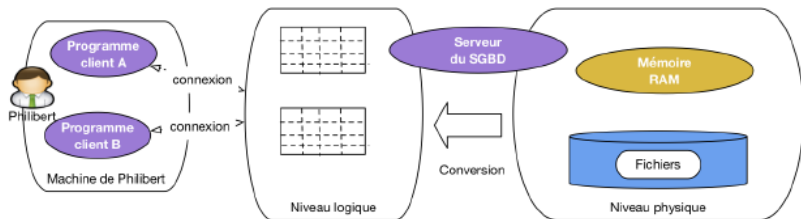
2 Rappels BD relationnelles

- Techniques implantées dans les systèmes relationnels
 - ⇒ **Méthodes de stockage** qui exploitent les ressources physiques de la machine pour assurer la disponibilité et la sécurité des bases de données;
 - ⇒ **Structures de données**, parfois sophistiquées, utilisées pour obtenir de très bonnes performances même en présence de très gros volumes;
 - ⇒ Algorithmes et protocoles à différents niveaux pour garantir un comportement robuste et efficace du système: **optimisation des requêtes**
- Le cours comprend trois grandes parties
 - ↪ Organisation physique des BD (Chapitre 2: Dispositifs de stockage)
 - ↪ Indexation (Chapitre 3: Structures d'index et Arbre B+)
 - ↪ Introduction aux méthodes et algorithmes d'optimisation (Chapitre 4: Introduction à l'évaluation et optimisation et Chapitre 5: Opérateurs Relationnels et PEP)

- **Base de données: BD:** grand ensemble de données
 - structurées
 - mémorisées sur un support permanent
- **Système de gestion de base de données (SGBD):** assure la gestion de l'ensemble des informations stockées dans une base de données
 - 1 **Accès** aux fichiers de la base, garantissant leur **intégrité**, contrôlant les opérations **concurrentes**, **optimisant** les recherches et mises à jour
 - 2 **Interactions** avec les applications et utilisateurs, grâce à des langages d'interrogation et de manipulation à haut niveau d'abstraction



Niveaux d'abstraction et modèle de données



- Le **niveau physique** : codage des données dans des fichiers stockés sur disque
- Le **niveau logique** : la représentation des données dans des structures abstraites, proposées aux applications clientes, obtenues par conversion du niveau physique
- Le niveau logique définit la modélisation des données. On peut envisager par exemple des structures de graphe, d'arbre, de listes, etc. Le modèle relationnel se caractérise par une modélisation basée sur une seule structure, **la table**

- Représentation abstraite des données, proposée par un SGBD
 - Reste indépendant de la façon dont les données sont réellement stockées et manipulés au niveau physique $\Rightarrow$ plus simple à utiliser
 - Modèles logiques: relationnel, objet, semi-structuré
- Langages offerts par le niveau logique
 - Langage de description de données (LDD): schémas de données utilisées
 - Langage de manipulation de données (LMD): modification des données
 - Langage de requêtes : interrogation des données

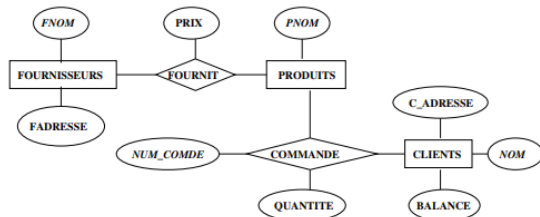
$\Rightarrow$ SGBD **relationnels** $\Rightarrow$ un seul langage pour les trois: **SQL**

- Correspond à la vision de l'utilisateur (de l'application) sur la BD
 - Plusieurs applications peuvent exister au-dessus des mêmes données
 - Chacune peut avoir besoin d'une vision différente de la base de données
- S'appuie sur le modèle conceptuel de données
- Niveau conceptuel des données
 - Description générale des données, indépendante d'un SGBD particulier
 - Modèles conceptuels: entité-association, UML, Merise, ...

- Représentation concrète (physique) des données dans une BD
 - Stockage sur disque
 - Structures d'indexation
- Opérations réalisées par le SGBD sur cette représentation physique pour implémenter les opérations logiques
 - Accès aux données, gestion sur disque des données / des index
 - Partage de données et gestion de la concurrence d'accès
 - Reprise sur pannes (fiabilité)
 - Distribution des données et interopérabilité (accès aux réseaux)

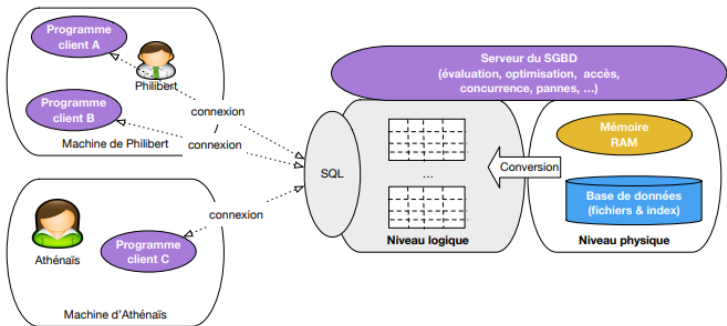
Exemple

- Modèle conceptuel (externe)



- Modèle logique (relationnel)
 - FOURNISSEUR (**FNOM**, FADRESSE)
 - CLIENT (**NOM**, C_ADRESSE, BALANCE)
 - FOURNITURE (**FNOM**, **PNOM**, PRIX)
 - COMMANDE (**NUM_COMDE**, **NOM**, **PNOM**, QUANTITE)
- Modèle physique (dépend du SGBD utilisé)
 - Fichiers sur disque, structures d'index
 - Algorithmes d'accès aux données, de modification, d'accès concurrent, etc.

Vue d'ensemble d'un système relationnel



- Indépendance entre le niveau logique et le niveau physique. Condition nécessaire de l'administration des SGBD

- **Stockage** : les données sont stockées de manière ordonnée sur des supports persistants
- **Indexation** : des structures de données permettent des accès très rapides à la base
- **Interrogation et mise à jour** : les requêtes de haut niveau d'abstraction (SQL) sont transcrites en algorithmes efficaces
- **Partage / Concurrency d'accès** : de très nombreuses applications peuvent accéder simultanément à une même base
- **Sécurité** : contrôle des accès et gestion des pannes logicielles ou matérielles

Rappels sur le modèle relationnel (par l'exemple)

- Les données sont structurées en tables (relations) et lignes (nuplets).
La ligne est l'unité d'information

- Relations: attributs typés (colonnes),
nuplets (lignes)
- BD = ensemble de relations
- Schéma relation = nom relation +
ensemble attributs avec leurs types
- Schéma BD = ensemble des schémas des
relations

idArtiste	nom	prénom	annéeNaissance
100	Eastwood	Clint	1930
101	Hackman	Gene	1930
102	Scott	Tony	1930
103	Smith	Will	1968

- Les données sont structurées et contraintes par un schéma

```
create table Artiste (idArtiste INTEGER NOT NULL,  
                      nom VARCHAR (30) NOT NULL,  
                      prénom VARCHAR (30) NOT NULL,  
                      annéeNaissance INTEGER,  
                      PRIMARY KEY (idArtiste));
```



Clé primaire, clé étrangère

Les tables sont liées par des valeurs partagées entre clé primaire et clé étrangère

idFilm	titre	année	genre	idRéalisateur	codePays
20	Impitoyable	1992	Western	100	USA
21	Ennemi d'état	1998	Action	102	USA

```
create table Film (idFilm integer NOT NULL,  
                  titre varchar (80) NOT NULL,  
                  année integer NOT NULL,  
                  genre varchar (20) NOT NULL,  
                  idRéalisateur integer,  
                  codePays varchar (4),  
                  primary key (idFilm),  
                  foreign key (idRéalisateur)  
                      references Artiste(idArtiste),  
                  foreign key (codePays)  
                      references Pays(code));
```

Requêtes relationnelles

- Algèbre relationnelle

- Ensemble d'opérateurs agissant sur des relations et produisant des relations
 - Sélection, projection, produit cartésien, jointure, différence, union, ...
 - Requête = composition d'opérateurs algébriques $\implies$ expression procédurale
- Ex. Les noms des propriétaires de voitures immatriculées avant 1989

$$\pi_{Proprietaire}(\sigma_{Annee < 1989}(VEHICULE))$$

- Calcul relationnel

- Expression déclarative des requêtes (formules logiques du premier ordre)
 - On dit ce que l'on veut obtenir, sans dire comment
 - Requête = formule logique
- Ex. $\{v.Proprietaire \mid VEHICULE(v) \wedge v.Annee < 1989\}$

- Théorie relationnelle: l'algèbre et le calcul sont équivalents

Important!

Le SGBD transforme une requête au niveau logique (SQL) en un programme d'évaluation algébrique au niveau physique

Langage de requêtes: SQL

- En pratique: langage SQL
 - Basé sur le calcul relationnel $\Rightarrow$ déclaratif
 - Puissant: ajout d'opérations sans équivalent dans l'algèbre (groupement, agrégation, tri, etc.)
- Ex.

```
SELECT v.Proprietaire  
FROM VEHICULE v  
WHERE v.Annee < 1989
```

- Le rapport SQL - algèbre
 - SQL: langage de haut niveau, déclaratif, adapté aux utilisateurs
 - Algèbre: langage procédural, adapté à l'exécution par le système
 - Principe :

les requêtes SQL sont traduites par le système en expression algébrique (plan d'exécution) et ensuite exécutées (après optimisation).



Le langage SQL : supposé connu!

- Une sélection

```
select titre  
from Film  
where année = 2016
```

- Une jointure

```
select titre, prénom, nom  
from Film as f, Artiste as a  
where f.idRéalisateur = a.idArtiste  
and année = 2016
```

↪ La méthode de calcul n'est jamais spécifiée

Algèbre relationnelle: supposée connue également!

- Opérateurs qui prennent une ou deux tables en entrée et produisent une table en sortie
 - Sélection (σ) : Restriction sur valeur des tuples
 - Projection (π) : Suppression de colonnes (allège le résultat)
 - Jointure ($\bowtie$) : Fusionne deux ensembles sur valeur
 - Différence ($-$) : Sous ensemble non présent dans le second
 - Union($\cup$) : Union des deux ensembles
- Les opérateurs sont composables pour construire des expressions

$$\pi_{titre}(\sigma_{annee=2016}(Film))$$

$$\pi_{titre,prenom,nom}(\sigma_{annee=2016}(Film) \bowtie_{idRealisateur=idArtiste} Artiste)$$

- Composition des sélections

$$\begin{aligned} & \sigma_{capacite > 100 \wedge type = 'Hotel'}(Logement) \\ & \equiv \\ & \sigma_{capacite > 100}(\sigma_{type = 'Hotel'}(Logement)) \end{aligned}$$

- L'union est l'équivalent en algèbre de la disjonction logique¹

$$\begin{aligned} & \sigma_{capacite > 100 \vee lieu = 'Corse'}(Logement) \\ & \equiv \\ & \sigma_{capacite > 100}(Logement) \cup \sigma_{lieu = 'Corse'}(Logement) \end{aligned}$$

Voyageur (**idVoyageur**, nom, prénom, ville, région)

Séjour (**idSéjour**, *idVoyageur*, *codeLogement*, début, fin)

Logement (**code**, nom, capacité, type, lieu)

Activité (**codeLogement**, **codeActivité**, description)

¹Si toute sélection avec un “ou” peut s’exprimer par une union, l’inverse n’est pas vrai ↻

Jointures: Nom et lieu des logements où l'on pratique la voile?

- Besoin des attributs *lieu* et *nom* qui apparaissent dans *Logement*, et de *codeActivité* qui apparaît dans *Activité*
 - Rapprocher les nuplets de *Logement* et de *Activité* $\Rightarrow$ une jointure
 - Déterminer le (ou les) attribut(s) sur lesquels se fait ce rapprochement $\Rightarrow$ **appariement** de la **clé primaire** d'une part (dans *Logement*), de la **clé étrangère** d'autre part (dans *Activité*)

$$\pi_{nom, lieu}(\text{Logement} \bowtie_{code=codeLogement} (\sigma_{codeActivité='Voile'}(\text{Activité})))$$

- La jointure reconstitue le lien entre des informations qui sont naturellement associées, mais qui ont été réparties dans plusieurs relations au moment de la conception de la base

- Nom des clients qui sont allés à Tabriz (en supposant connu le code, *ta*, de cet hôtel)
- Quels lieux a visité le client 30
- Nom des clients qui ont eu l'occasion de faire de la voile

Voyageur (**idVoyageur**, nom, prénom, ville, région)

Séjour (**idSéjour**, *idVoyageur*, *codeLogement*, début, fin)

Logement (**code**, nom, capacité, type, lieu)

Activité (**codeLogement**, **codeActivité**, description)

- Nom des clients qui sont allés à Tabriz

$$\pi_{nom}(Voyageur \bowtie_{idVoyageur=idVoyageur} (\sigma_{codeLogement='ta'}(Séjour)))$$

- Quels lieux a visité le client 30

$$\pi_{lieu}(\sigma_{idVoyageur=30}(Séjour) \bowtie_{codeLogement=code} Logement)$$

- Nom des clients qui ont eu l'occasion de faire de la voile. En deux étapes:

$$R1 := Séjour \bowtie_{codeLogement=codeLogement} (\sigma_{codeActivité='Voile'}(Activité))$$

$$\pi_{nom}(Voyageur \bowtie_{idVoyageur=idVoyageur} R1)$$